deployment/kubernetes/ applications/catalog-service/staging). The first thing to configure is a reference to the base manifests.

If you've followed along, you should have your Catalog Service source code tracked in a `catalog-service` repository on GitHub. A reference to a remote base needs to point to the folder containing the kustomization.yml file, which is k8s in our case. Also, we should refer to a specific tag or digest for the version we want to deploy. We'll talk about release strategies and versioning in the next chapter, so we'll simply point to the `main` branch for now. The final URL should be something like `github.com/<your_github_username>/catalog-service/k8s?ref=main`. For example, in my case, it would be `github.com/polarbookshop/catalog-service/k8s?ref=main`.

Listing 14.10 Defining an overlay for staging on top of a remote base

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:                                                    ❶
  - github.com/<your_github_username>/catalog-service/k8s?ref=main
```

❶ Uses the manifests in your Catalog Service repo on GitHub as the base for further customizations

NOTE I'll assume that all the GitHub repositories you created for Polar Bookshop are publicly accessible. If that's not the case, you can go to the specific repository page on GitHub and access the Settings section for that repository. Then scroll to the bottom of the settings page, and make the package public by clicking the Change Visibility button.

We could now deploy Catalog Service from the staging overlay using the Kubernetes CLI, but the result wouldn't be different than using the base directly. Let's start applying some customizations specifically for staging deployments.